

Báo Cáo: Cơ Chế Load Video — VideoDetailPage

Mục Lục

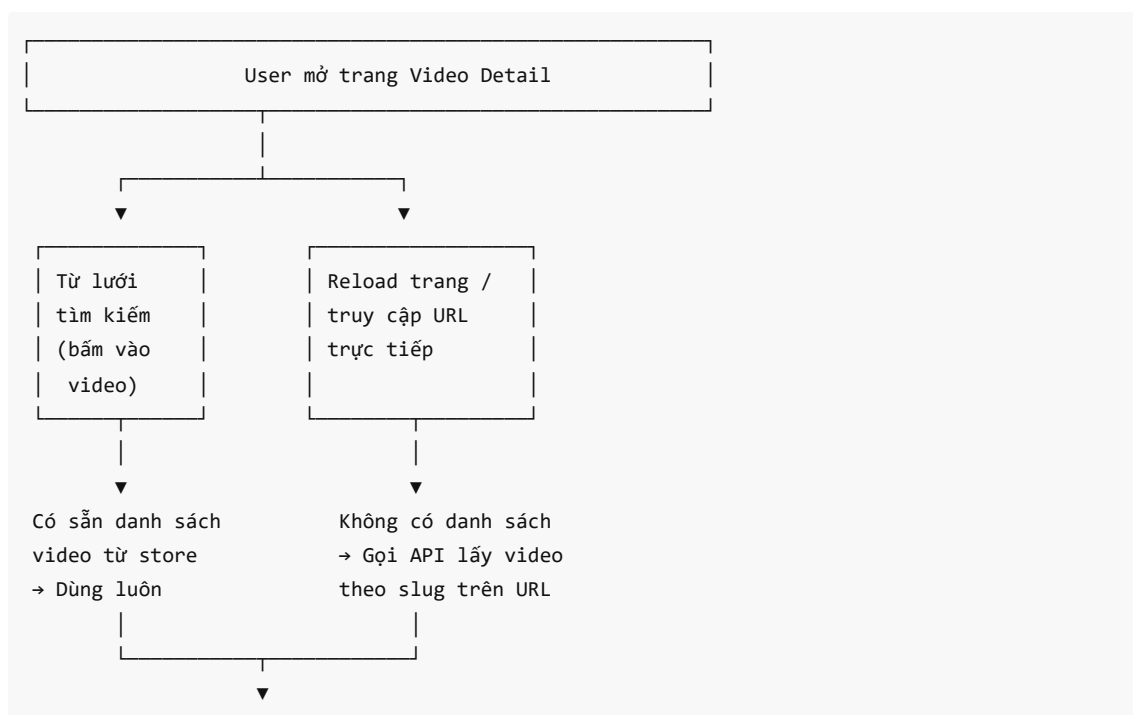
- [1. Tổng Quan](#)
- [2. Hai Luồng Vào Trang](#)
- [3. Cơ Chế "Pool" Video Element](#)
- [4. Cơ Chế Lazy Load theo Vùng Nhìn \(Viewport\)](#)
- [5. Cơ Chế Preload Video Tiếp Theo](#)
 - [Preload hoạt động khác nhau giữa Android và iOS](#)
 - [Tại sao Android preload được đúng 2 video?](#)
 - [Giới Hạn Preload trên iOS](#)
- [6. Vô Tận \(Infinite Scroll\)](#)
- [7. "Reload Gate" — Cơ Chế Khóa Khi Tải Lại Trang](#)
- [8. Tắt Tắt Luồng Hoàn Chính](#)
- [9. So Sánh Web App và Native App](#)
- [10. Giải Thích Nhanh Các Khái Niệm Kỹ Thuật](#)

1. Tổng Quan

Trang video detail cho phép user lướt dọc qua từng video, mỗi video chiếm toàn màn hình. Hệ thống được thiết kế để:

- Video tiếp theo **đã sẵn sàng trước** khi user lướt tới
- Không lãng phí tài nguyên cho video xa
- Xử lý khác biệt giữa truy cập từ **lưới tìm kiếm** và truy cập **trực tiếp / tải lại trang**

2. Hai Luồng Vào Trang



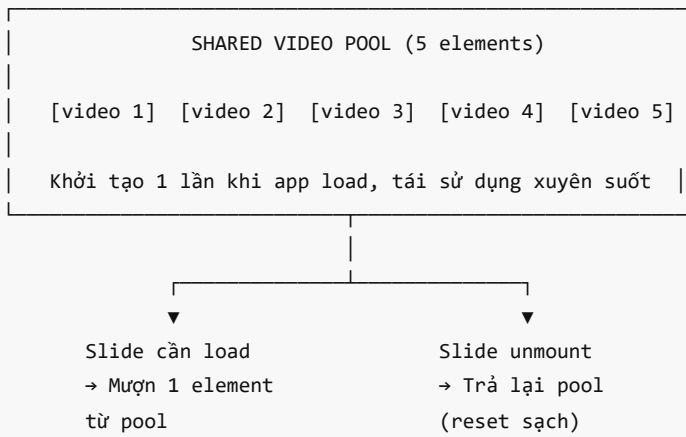
Render danh sách video
(vô tận – infinite scroll)

Luồng 1 — Từ lưới tìm kiếm: Danh sách video đã có sẵn trong bộ nhớ (Zustand store). Trang mở ngay lập tức, video đang xem là video user vừa bấm vào.

Luồng 2 — Reload / URL trực tiếp: Không có danh sách sẵn. Hệ thống gọi API để lấy video theo URL slug, sau đó tiếp tục fetch thêm video để người dùng có thể vuốt.

3. Cơ Chế "Pool" Video Element

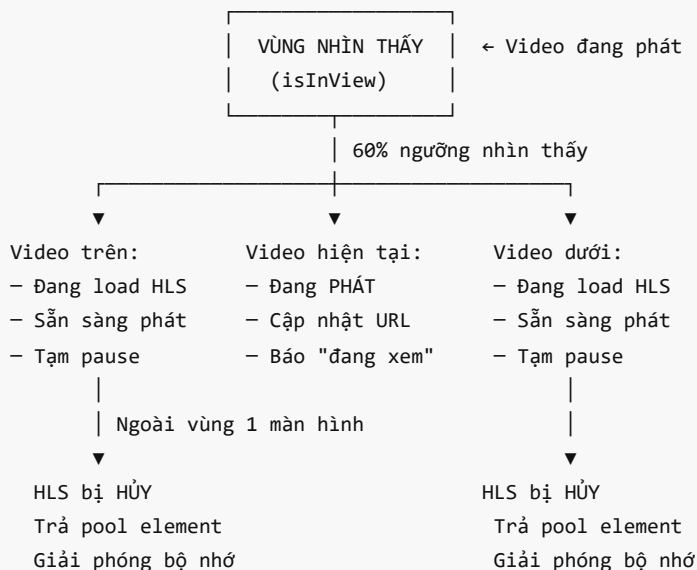
Đây là kỹ thuật cốt lõi để đảm bảo hiệu năng, đặc biệt trên **iOS Safari**.



Tại sao cần Pool? iOS Safari cấp phép phát video có tiếng chỉ khi user **chạm vào màn hình**. Việc reuse element đã được "unlock" giúp video tiếp theo phát có tiếng ngay mà không cần user tap thêm.

4. Cơ Chế Lazy Load theo Vùng Nhìn (Viewport)

Không phải tất cả video đều load cùng lúc. Hệ thống chia 3 vùng:



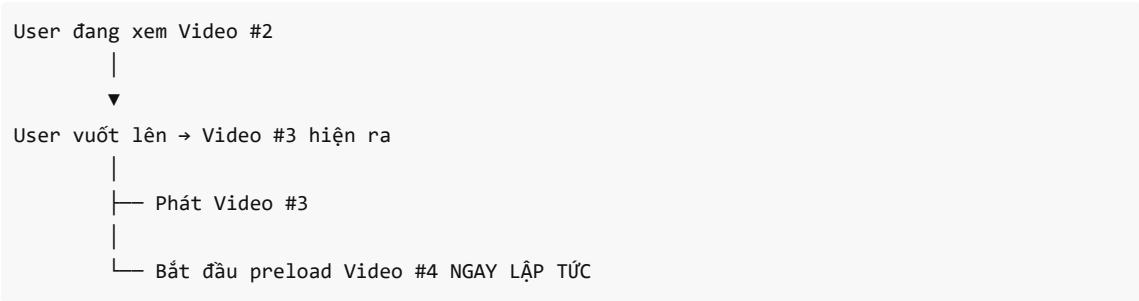
Quy tắc kích hoạt:

Khoảng cách	Trạng thái
Trong vòng ~1 màn hình	Load HLS, sẵn sàng phát
Đang nhìn thấy (>60%)	Phát video + cập nhật URL
Ngoài 1 màn hình	Hủy load, giải phóng tài nguyên

5. Cơ Chế Preload Video Tiếp Theo

Khi user vuốt sang video mới, hệ thống **ngay lập tức** preload video kế tiếp để loại bỏ stutter (giật lag) ở lần vuốt sau.

Ngay khi danh sách video load xong lần đầu, hệ thống cũng preload sẵn video ngay kế tiếp — nhằm đảm bảo lần vuốt đầu tiên không bị giật.

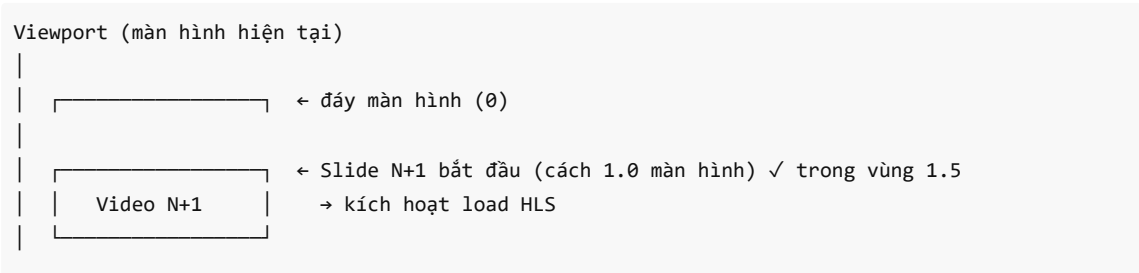


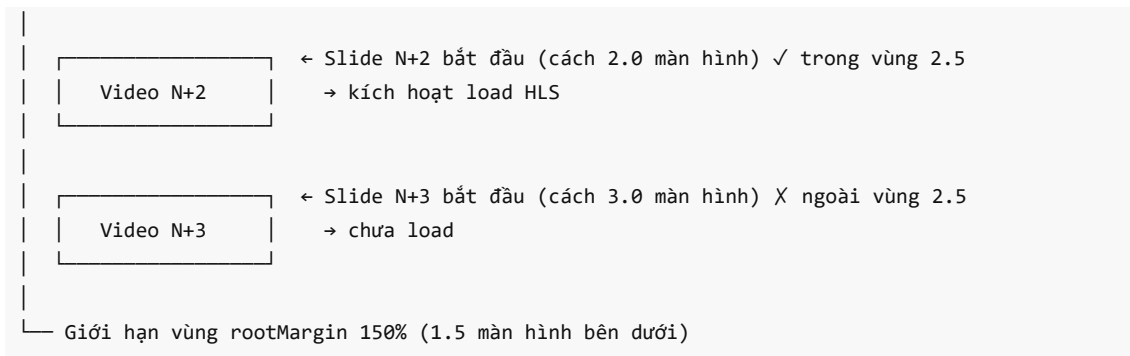
Preload hoạt động khác nhau giữa Android và iOS

	Android	iOS
Cơ chế	hls.js autoStartLoad:true — khi slide vào gần màn hình, fragment HLS tự load ngay	Thử fetch segment vào cache trình duyệt + gọi play() silent để AVFoundation buffer
Số video preload trước	2 video phía trước (và 2 phía sau)	Chưa đảm bảo — xem giới hạn bên dưới
Hiệu quả	Hoạt động ổn định	Chưa đảm bảo
Kết quả	Video N+1, N+2 đã có buffer khi user vuốt tới	Video kế tiếp có thể vẫn phải load lại từ đầu

Tại sao Android preload được đúng 2 video?

Mỗi slide cao bằng 1 màn hình (100dvh). Cơ chế `isNearView` dùng `rootMargin: 150%` — nghĩa là vùng theo dõi mở rộng thêm 1.5 màn hình ra ngoài vùng nhìn thấy:





Khi user đang xem Video N, hls.js của N+1 và N+2 đã tự động load fragment và buffer sẵn **tối đa 10 giây** mỗi video (`maxBufferLength: 10`). Khi vượt tới N+1 → phát gần như tức thì.

Giới Hạn Preload trên iOS

iOS Safari có 2 ràng buộc khiến preload chưa thực sự hoạt động:

1. `play()` bị Safari chặn nếu không có user gesture

Để AVFoundation thực sự buffer dữ liệu, cần gọi `play()` trên video element. Nhưng Safari chỉ cho phép `play()` khi có **cử chỉ thực của người dùng** (tap/click):

Reload flow:

- Pool chưa được unlock (user chưa tap Play)
- preload gọi `play()` trên video #4
- Safari REJECT
- Không có buffer → khi vượt tới vẫn load từ đầu

From-grid flow:

- Pool đã unlock (user tap vào video từ lưới)
- preload gọi `play()` trên video #4
- Safari ACCEPT (element đã có gesture token)
- Có buffer → vượt tới phát nhanh hơn (nhưng không đảm bảo 100%)

2. iOS chủ động throttle băng thông cho media element không hiển thị

Đây là ràng buộc từ chính hệ điều hành iOS, không phải lỗi code. WebKit/AVFoundation áp dụng chính sách: **chỉ tải dữ liệu cho video đang thực sự phát**, các video element khác dù đã có `src` và gọi `load()` vẫn bị throttle hoặc hoàn toàn không được tải network.

Android (Chrome + hls.js):

- hls.js tự quản lý tải fragment qua XMLHttpRequest
- Không phụ thuộc vào quyết định của trình duyệt
- Slide N+1, N+2 load fragment song song khi vào gần viewport ✓

iOS (Safari + AVFoundation):

- AVFoundation quyết định khi nào tải dữ liệu
- Video không đang `play()` → AVFoundation từ chối tải thêm segment
- Gọi `load()` / `set src` cũng không giúp ích
- Chỉ `play()` mới ép AVFoundation bắt đầu buffer ✗ (bị chặn bởi gesture)

3. Fetch vào NSURLCache không chắc chắn được AVFoundation dùng

Cách dự phòng là fetch các file `.m3u8` / `.ts` vào cache HTTP của trình duyệt để khi AVFoundation request cùng URL sẽ gặp cache hit. Tuy nhiên AVFoundation có thể dùng cache riêng tách biệt khỏi NSURLCache của WKWebView — khiến cách này không đáng tin cậy.

Tóm lại: Preload cho iOS đang được cố gắng hỗ trợ nhưng **chưa hoạt động ổn định** do 3 lớp ràng buộc chồng nhau: *gesture requirement* → *bandwidth throttle* → *cache isolation*. Người dùng iOS có thể thấy loading spinner khi vuốt sang video mới, đặc biệt trong *reload flow*.

6. Vô Tận (Infinite Scroll)

Danh sách video: [1] [2] [3] [4] [5] [6] [7] ...

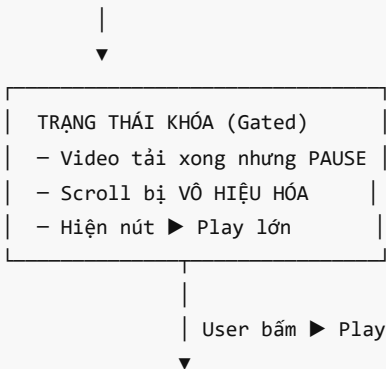
↑
Còn 2 video nữa
→ Tự động gọi API
fetch thêm video

Khi user đang xem video còn cách cuối danh sách **2 video**, hệ thống tự động gọi API để tải thêm. User không bao giờ thấy "hết video".

7. "Reload Gate" — Cơ Chế Khóa Khi Tải Lại Trang

Đây là hành vi **chỉ xảy ra khi reload trang hoặc truy cập URL trực tiếp** (không phải từ lưới tìm kiếm):

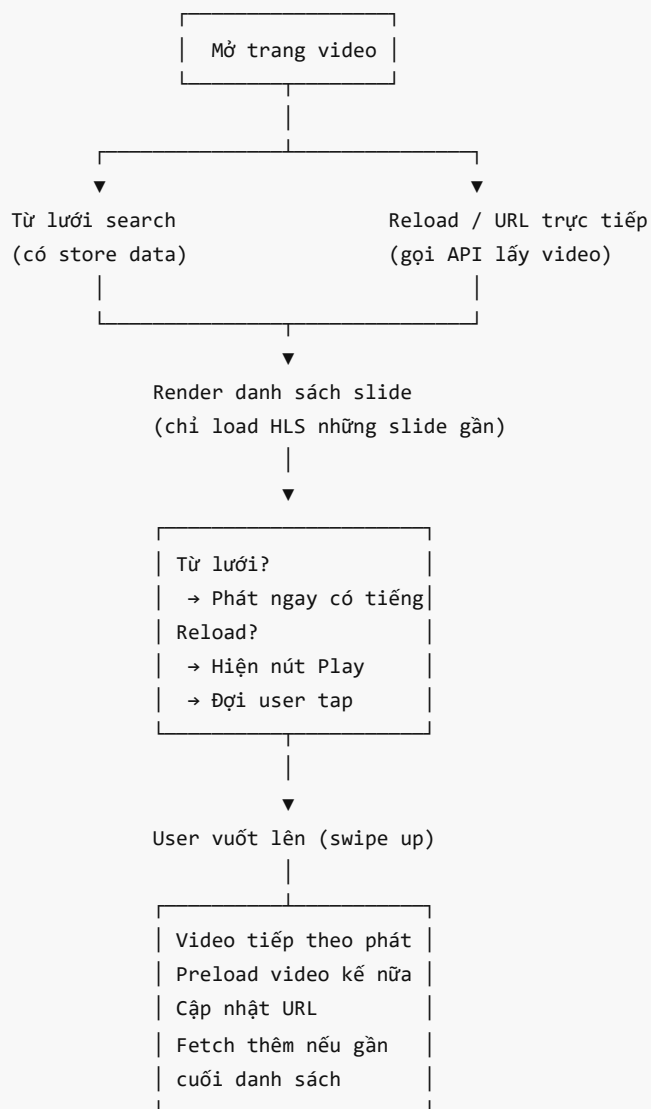
User reload trang / paste URL



- Video bắt đầu phát CỐ TIẾNG
- Scroll được bật lại
- Unlock toàn bộ video pool (iOS Safari được phép phát tiếng cho tất cả video)

Tại sao cần cơ chế này? iOS Safari yêu cầu **cử chỉ thực của người dùng** (tap/click) để phát video có tiếng. Khi reload, không có cử chỉ nào được ghi nhận → bắt buộc user phải tap Play một lần để "unlock" âm thanh cho toàn bộ session.

8. Tóm Tắt Luồng Hoàn Chính



9. So Sánh Web App và Native App

Phần này giải thích tại sao cùng một tính năng TikTok-style nhưng trải nghiệm trên Web App (hiện tại) và Native App (iOS/Android thuần) lại khác nhau đáng kể.

Preload & Buffering

Khả năng	Native App (Swift/Kotlin)	Web App (hiện tại)
Preload video kế tiếp trên Android	✅ Hoàn toàn chủ động	✅ Hoạt động tốt qua hls.js
Preload video kế tiếp trên iOS	✅ Hoàn toàn chủ động	⚠️ Chưa ổn định
Số video buffer trước	Tùy config, thường 2–3	Android: 2, iOS: không đảm bảo
Phát ngay khi vuốt (no stutter)	✅ Gần như 100%	✅ Android / ⚠️ iOS

Native App trên iOS dùng `AVQueuePlayer` hoặc `AVPlayerViewController` — Apple cung cấp API riêng cho phép preload nhiều video song song mà không cần user gesture, không bị throttle băng thông.

Web App bị ràng buộc bởi chính sách của Safari/WebKit: mọi media phải có gesture mới được phát, và iOS chủ động throttle băng thông cho video không đang hiển thị.

Âm Thanh (Audio Autoplay)

Tình huống	Native App	Web App
Mở app lần đầu → video có tiếng ngay	✓	✗ Cần user tap
Reload trang → video có tiếng ngay	✓	✗ Hiện nút Play, chờ tap
Vuốt sang video mới → tiếng liên tục	✓	✓ (sau khi unlock lần đầu)

Web App phải dùng **Reload Gate** (cơ chế khóa buộc user tap Play) để bypass giới hạn autoplay của trình duyệt — không cần thiết trên Native App vì OS không áp đặt ràng buộc này.

Quản Lý Bộ Nhớ & Hardware Decoder

	Native App	Web App
Số decoder video song song	Tùy RAM thiết bị	Bị giới hạn bởi trình duyệt (thường 4–5)
Giải phóng decoder	Chủ động, tức thì	Cần <code>removeAttribute('src')</code> + <code>load()</code> mới thực sự trả
Pool video element	Không cần (OS quản lý)	Bắt buộc phải tự implement (5 elements)

Web App phải tự implement **Video Pool** (5 element tái sử dụng) để không vượt quá giới hạn hardware decoder của iOS — Native App không có vấn đề này vì OS cấp phát decoder linh hoạt hơn.

Tóm Tắt Khoảng Cách Trải Nghiệm

Tính năng	Native App	Web Android	Web iOS
Phát ngay khi mở	✓	✓	⚠ (gate)
Preload trước khi vuốt	✓	✓	⚠
No-stutter khi vuốt	✓	✓	⚠
Autoplay có tiếng	✓	✓	✗
Bộ nhớ hiệu quả	✓	✓	✓ (pool)

Kết luận cho BA: Web App trên Android đã đạt trải nghiệm gần với Native App. Khoảng cách lớn nhất còn lại là **iOS**, nơi Safari áp đặt các ràng buộc về gesture và băng thông mà Web App không thể vượt qua hoàn toàn nếu không có Native App wrapper (WKWebView trong Swift vẫn bị một số ràng buộc tương tự).

10. Giải Thích Nhanh Các Khái Niệm Kỹ Thuật

Thuật ngữ	Ý nghĩa cho BA
-----------	----------------

HLS	Định dạng video streaming (như YouTube dùng), tải từng đoạn nhỏ thay vì tải cả file
Video Pool	5 "khung video" được tạo sẵn và tái sử dụng — tiết kiệm bộ nhớ và tránh giới hạn iOS
IntersectionObserver	Công nghệ theo dõi element có đang trong tầm nhìn hay không — không cần tính toán tay
Gated / ForcePause	Trạng thái khóa bắt buộc user phải tap để iOS cho phép phát tiếng
Infinite Scroll	Tự động tải thêm video khi gần cuối — user không bao giờ thấy hết
Shallow Route Replace	Cập nhật URL khi vuốt mà không reload trang
AVFoundation / AVQueuePlayer	Engine video của Apple — Native App dùng trực tiếp, Web App chỉ truy cập gián tiếp qua Safari
Bandwidth Throttle	iOS chủ động giới hạn băng thông cho video không đang phát — Native App không bị ràng buộc này